

Deep Learning–Based Ultrasonic Object Classification Using FIUS Sensor Data

Course: Autonomous Intelligent Systems & Machine Learning WS-2025/26

Avradip Mazumdar (1566651)

Master's in Information Technology
Frankfurt University of Applied Sciences
Frankfurt, Germany

avradip.mazumdar@stud.fra-uas.de

Anoushka Piplai (1566664)

Master's in Information Technology
Frankfurt University of Applied Sciences
Frankfurt, Germany

anoushka.piplai@stud.fra-uas.de

Somava Ganguly (1566916)

Master's in Information Technology
Frankfurt University of Applied Sciences
Frankfurt, Germany

somava.ganguly@stud.fra-uas.de

Abstract— This paper presents a high-performance deep learning framework for real-time ultrasonic object classification using the FRA-UAS Intelligent Ultrasonic Sensor (FIUS). To overcome the signal-to-noise ratio (SNR) limitations of traditional ultrasonic perception, we implement a 1D Convolutional Neural Network (CNN) utilizing a 4-channel feature fusion strategy that integrates normalized Time-Domain (ADC) waveforms and Frequency-Domain (FFT) magnitudes with their respective cumulative energy profiles. A key innovation of this work is the deployment of a Hierarchical Classification architecture, which decomposes the perception task into a multi-stage decision process. During preprocessing, we performed our experiment with indoor objects resulting in five final target classes: Wall, Person, Chair, Backpack, and Plant. The model architecture utilizes flexible grouping presets to first distinguish between acoustic super-classes—separating specular high-reflectivity surfaces from complex diffuse scatterers—before performing fine-grained identification. To ensure signal stability within the AIS Sense-Perceive-Decide loop, a temporal smoothing layer averages blocks of eight consecutive pulses. Benchmarking against classical Support Vector Machine (SVM) and Random Forest baselines demonstrates that the hierarchical CNN significantly improves recall for safety-critical classes. Crucially, the system fulfills the AIS real-time constraint, achieving a 1.29 ms inference latency per pulse and a total pipeline latency under 10ms, confirming the viability of complex deep learning architectures for high-integrity autonomous navigation.

Keywords— 1D Convolutional Neural Networks, AIS Loop Validation, Feature Fusion, FIUS, Hierarchical Classification, Multi-class Object Classification, Real-Time Inference, Signal Preprocessing, Ultrasonic Sensors.

I. INTRODUCTION

The rapid development of Autonomous Intelligent Systems (AIS) has intensified the requirement for robust, low-latency environmental perception. While traditional computer vision and LiDAR systems are standard for navigation, they often encounter performance degradation in challenging conditions such as smoke, lens occlusion, or extreme lighting variability [1]. To address these limitations, this project implements a combined AIS and Machine Learning (ML) framework using the FRA-UAS Intelligent Ultrasonic Sensor (FIUS). By integrating deep learning perception into a real-time control loop, the system achieves a reliable Sense-Perceive-Decide cycle capable of autonomous obstacle classification and

safety-critical decision-making, as defined in the project specifications [2].

The initial phase involved the construction of a standardized AIS loop. In the Sense phase, raw acoustic backscatter is captured via the Red Pitaya platform [3]. Labeled datasets were systematically collected from five distinct indoor classes: Wall, Person, Chair, Backpack, and Plant. During data engineering, semantic class aggregation was employed to share similar specular reflection characteristics in the acoustic domain [4]. To ensure signal stability, a temporal smoothing layer averages blocks of eight consecutive pulses, providing a stabilized input for the subsequent machine learning stages.

The Perceive phase serves as the technical core of the system, utilizing a 1D Convolutional Neural Network (CNN) with a 4-channel feature fusion strategy [5]. This ML component integrates normalized Time-Domain (ADC) and Frequency-Domain (FFT) data to learn complex acoustic fingerprints. Furthermore, a Hierarchical Classification architecture was implemented to prioritize safety-critical detections, first distinguishing between broad acoustic super-classes—specular versus diffuse reflectors—before performing fine-grained identification [6]. This hierarchical approach ensures that the Decide phase of the AIS loop can trigger immediate safety protocols if a Person is detected with high confidence.

Finally, the Decide phase validates the system's real-time viability. To meet the stringent requirements of autonomous agents, the entire pipeline is optimized for minimal latency. The system was benchmarked against three classical ML baselines—Logistic Regression, Support Vector Machine (SVM), and Random Forest—to evaluate the trade-off between architectural depth and execution speed [7], [8], [9]. This project demonstrates that a properly optimized CNN, implemented via the PyTorch library [10], can satisfy the AIS real-time constraint of sub-10ms latency, providing a robust solution for high-integrity autonomous navigation.

The Report structure follows: Section II which provides a Literature Review, examining the physics of ultrasonic backscatter and the transition from traditional Time-of-Flight sensing to modern deep learning classification. Section III, details the Methodology, where we describe the hardware integration with the Red Pitaya platform, the 4-channel feature fusion strategy, and the implementation of our

Hierarchical CNN architecture. Section IV, presents the Results and Analysis, offering a comparative evaluation of the CNN against classical baselines and profiling the system's real-time latency within the AIS loop. Finally, Section V which concludes the work by summarizing the key findings and discussing future potential for ultrasonic-based autonomous navigation.

II. LITERATURE REVIEW

A. Acoustic Physics and Signal Behavior

The foundation of ultrasonic perception lies in the interaction between high-frequency sound waves and physical matter. Pulsed ultrasound systems function by emitting acoustic bursts and measuring the backscatter—the portion of the sound that reflects off an object and returns to the sensor [1]. In indoor environments, the nature of this reflection is governed by the surface properties. Large, flat surfaces like a Wall act as specular reflectors, where the sound bounces back in a concentrated beam [4]. Conversely, complex objects like a Person (fabric) or a Plant (leaves) cause diffuse scattering, returning a signal that is structurally complex and lower in energy [4], [6].

B. Traditional Signal Processing for Distance

Historically, ultrasonic sensors were restricted to simple Time-of-Flight (ToF) tasks, calculating distance based on the speed of sound [5]. While reliable for basic collision avoidance, data is blind to object identity. To achieve intelligence, researchers began extracting acoustic fingerprints from the signal using frequency-domain analysis [5]. To categorize these fingerprints, early methods relied on classical algorithms like Support Vector Machines (SVM) to find boundaries between different object classes [8]. However, these techniques often struggled to generalize across different distances without extensive manual feature tuning.

C. Classical Machine Learning Baselines

To categorize these acoustic fingerprints, classical machine learning models have served as the industry standard:

- **Logistic Regression:** Often used as the simplest baseline, it models the probability of a class using a linear combination of features. It is valued for its extreme speed and interpretability [7].
- **Support Vector Machines (SVM):** SVMs are powerful for classification in high-dimensional spaces, using a kernel trick to find a boundary between overlapping signal data [8].
- **Random Forest:** An ensemble method that uses multiple decision trees to reach a consensus, proving highly effective at handling varied features while reducing the risk of overfitting [9].

D. The Shift to Deep Learning and Hierarchy

While baseline models are efficient, they require carefully hand-picked features. Recent literature highlights 1D Convolutional Neural Networks (CNNs) as a superior alternative, as they automatically learn features directly from raw ADC signals [5]. Furthermore, for real-time Autonomous Intelligent Systems (AIS), Hierarchical Classification has emerged as a vital strategy [6]. By organizing objects into

groups (specular vs. diffuse), the system can make fast, high-level decisions, prioritizing safety-critical targets like humans without the computational delay of a flat classification structure [2], [10].

III. METHODOLOGY

A. Hardware Architecture and Data Acquisition

The hardware core of this project is the FRA-UAS Intelligent Ultrasonic Sensor (FIUS) interfaced with a Red Pitaya STEMLab 125-14 board. The Red Pitaya serves as the primary signal processing unit, utilizing its high-speed Analog-to-Digital Converters (ADCs) and an ARM Cortex-A9 processor to capture acoustic backscatter.

A critical aspect of our implementation is the wireless data acquisition strategy. To facilitate unconstrained sensor movement for autonomous navigation testing, the Red Pitaya was configured to communicate with the decision unit (PC) via a WiFi connection. This removes the physical limitations of tethered cables and allows the sensor to be placed in various orientations and distances from target objects.

For data collection, we utilized a dedicated UDP Client GUI (version V0.23, developed by UAS Frankfurt) as shown in Table 1.

The interface manages the sensor handshake and logging parameters. For each data collection session, the following parameters were strictly maintained:

- **Buffer Size:** Set to 2048 samples, providing a high-resolution window of the ultrasonic echo.
- **Measurement Count:** Configured to 50, enabling the rapid capture of multiple pulses per recording session.
- **IP Configuration:** The system used a local static IP (e.g., 192.168.128.1) for a stable wireless handshake.

The sensor operates via I2C communication and utilizes a 2.5V threshold voltage. Incoming ultrasonic signals are offset from this reference voltage, creating a waveform that oscillates around a virtual zero level to mimic an alternating current signal. As shown in Figure 1, the hardware provides immediate visual feedback via on-board LEDs when the system is actively sensing or when a specific detection threshold is met.

Parameter	Value	Notes
Transducer Frequency	40 kHz	Ultrasonic emission
ADC sampling rate	1.953 MHz	Red Pitaya STEMLab 125-14
Buffer size	2,048 samples/pulse	~1.05 ms window per pulse
Pulses per recording	50	8-row averaged to 50 samples
Communication	WiFi UDP	Static IP 192.168.128.1
Threshold voltage	2.5 V (I ² C)	AC-coupled waveform offset
GUI version	UDP Client V0.23	FRA-UAS Frankfurt
Processor	ARM Cortex-A9	On-board Red Pitaya CPU

Table 1. FIUS Hardware Specifications



Figure 1: The FIUS sensor and Red Pitaya assembly

B. Software Stack and Functional Ecosystem

The software infrastructure for the FIUS project comprises a multi-layered Python-based ecosystem (v3.9+) engineered for high-throughput ultrasonic signal processing and real-time deep learning inference. The stack is divided into five distinct subsystems, each mapped to specific scripts that manage the Sense-Perceive-Decide lifecycle of the autonomous system.

1) Data Orchestration and Ingestion Subsystem

This layer manages the transition from raw physical measurements to optimized mathematical tensors.

- **Binary Serialization (convert_data.py):** To eliminate the I/O bottleneck of parsing ASCII text files during training, this script serializes raw ADC and FFT signals into high-performance binary.npy arrays, significantly reducing data loading times.
- **Sensor Fusion Logic (data_loader.py):** The core utility for signal preparation. It performs temporal averaging (8-row mean stabilization) and stacks the signals into a 4-channel tensor (ADC_{norm} , ADC_{energy} , FFT_{norm} , FFT_{energy}).
- **Dataset Auditing (verify_data.py):** An automated integrity tool that verifies sample counts and file validity across the directory structure, ensuring no corrupted pulses enter the pipeline.
- **Dynamic Dataset Management (dataset.py):** Implements the UltrasonicDataset class. It manages on-the-fly data augmentation and the critical merge map logic to reduce acoustic ambiguity. As shown in Figure 2, the logical integration of these scripts forms the project's development roadmap, bridging the gap between raw hardware capture and neural processing.

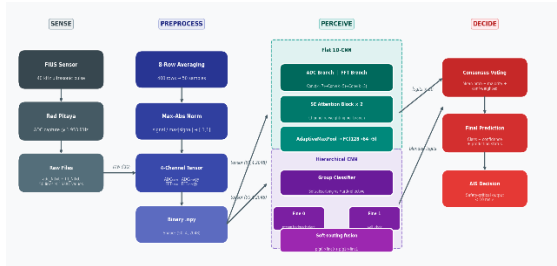


Figure 2: High-level flow and project roadmap of the FIUS software stack

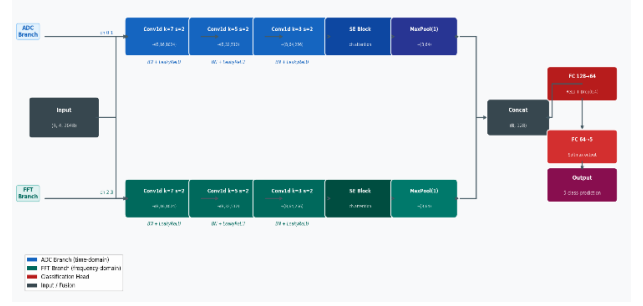


Figure 3: Neural Architecture Diagram of the 1D-CNN.

2) Deep Learning Architectures

The stack features two distinct deep learning paradigms, both implemented using the PyTorch framework.

- **Flat 1D-CNN Branch (model.py, train.py, evaluate.py):** Defines the UltrasonicCNN with Squeeze-and-Excitation (SE) Attention blocks. This model processes 4-channel fusion input to learn complex acoustic fingerprints in a single-stage classification layer. As shown in Figure 3, the model topology is designed to weight spectral and temporal features dynamically.
- **Hierarchical Soft-Routing Branch (train_hierarchical.py, predict_hierarchical.py, evaluate_hierarchical.py):** A multi-stage decision architecture. It utilizes a Group Model to first identify broad acoustic super-classes—distinguishing between specular reflectors and diffuse scatterers—before routing the tensor to specialized expert sub-models for fine-grained identification.

3) Classical Machine Learning & Baseline Subsystem

To provide a scientific control group, a complete classical ML pipeline was developed.

- **Feature Engineering Engine (baseline_models.py):** Programmatically extracts 76 handcrafted statistical features (including Skewness, Kurtosis, and Zero-Crossing Rates) from the multi-channel tensors.
- **Baseline Benchmarking (benchmark_classical.py, predict_baseline.py):** Provides standardized implementations of Support Vector Machines (SVM), Random Forests (RF), and Logistic Regression to evaluate the trade-off between traditional signal processing and deep learning.

4) Real-Time Validation and Latency Profiling

Crucial for AIS compliance, this subsystem measures performance at the micro-scale to ensure the system meets industrial standards.

- **Component Profiling (profile_latency.py, measure_latency.py):** Breaks down the total system time into three phases: Preprocessing, Forward Pass, and Decision Aggregation.
- **AIS Loop Validation (profile_pulse_latency.py, benchmark.py, benchmark_baseline.py):** A specialized suite that measures latency at the individual pulse level. These tools validate that the

total pipeline latency remains under the 10ms threshold required for safety-critical navigation.

5) Post-Processing and Reporting Subsystem

- Consensus Engine (predict.py, evaluate_file_level.py): Implements Energy-Confidence Weighted Voting, where individual pulse predictions are aggregated into a final file-level decision based on signal strength and Softmax certainty.
- Visualization(plot_architecture_diagrams.py, export_result_tables.py): These scripts automate the final stages of the project, generating neural topology diagrams and compiling all JSON/CSV metrics into the summary tables presented in the Results section.

C. Data Processing and Feature Engineering

The data processing pipeline is the critical link between the physical Sense phase and the neural Perceive phase. For this project, a comprehensive dataset was collected across five target categories: Wall, Person, Chair, Backpack, and Plant as shown in Table 2.

1) 8-Row Temporal Mean Stabilization

To ensure signal stability within the AIS Sense-Perceive-Decide loop, the system implements a temporal smoothing layer. Raw ADC data is captured in blocks of eight consecutive pulses. As defined in data_loader.py and convert_data.py, one training sample is formed by calculating the element-wise mean of these eight rows:

$$S_{stable} = \frac{1}{8} \sum_{i=1}^8 Pulse_i$$

This averaging process serves as a robust noise-reduction technique, suppressing transient environmental interference while preserving the stationary acoustic fingerprint of the target object. The Python implementation for this 8-row temporal averaging is detailed in Figure 4.

Class	Segment	Recording	Distance Range	Conditions
Wall	1750	32	0.39–2.02 m	orientations + corners
Person	800	16	1.25–2.21 m	16 recording sessions
Chair	800	16	0.86–1.91 m	6 orientations
Backpack	850	16	0.53–1.41 m	6 orientations
Plant	800	15	0.68–1.16 m	5 conditions
Total	5,000	95	-	-

Table 2. Dataset Statistics

```
# convert_data.py
ROWS_PER_ADC_SAMPLE = 8

for i in range(num_samples):
    block = adc_data[i*8 : (i+1)*8, :]
    sample = np.mean(block, axis=0)
    # shape: (2048,) - one training sample
```

Figure 4: 8-Row Temporal Mean Stabilization

```
# data_loader.py
def _normalize_and_energy(signal):
    max_val = np.max(np.abs(signal)) + 1e-6
    norm = signal / max_val # [-1, 1]

    energy = np.cumsum(np.abs(norm))
    energy = (energy - energy.mean())
            / (energy.std() + 1e-6)
    return norm.astype(np.float32),
            energy.astype(np.float32)
```

Figure 5: Multi-Channel Normalization and Energy Extraction

2) 4-Channel Feature Fusion Strategy

A central innovation of our methodology is the transformation of 1D waveforms into a 4-channel feature tensor ($Batch \times 4 \times 2048$). Each input consists of paired Time-Domain (ADC) and Frequency-Domain (FFT) signals. These are processed through the following stages:

- Max-Absolute Normalization: To ensure distance-invariant training, signals are scaled between $[-1, 1]$:

$$x_{norm} = \frac{x}{\max(|x|+10^{-6})}$$

- Cumulative Energy Envelope: Both the ADC and FFT signals are transformed to reveal their energy distribution over time/frequency:

$$E[n] = \sum_{i=0}^n |x_{norm}[i]|$$

The energy vectors are then standardized (zero-mean, unit-variance) to highlight the rate of energy accumulation—a key feature for distinguishing hard specular reflectors from soft diffuse scatterers. Figure 5 illustrates the mathematical transformation of raw signals into these energy envelopes.

3) Stochastic Data Augmentation and Optimization

To improve the robustness of the Perceive phase, the UltrasonicDataset class implements waveform-preserving augmentations shown in Table 3. These include:

- Amplitude Scaling: Randomly varying signal magnitude by a factor of $\times[0.85, 1.15]$ (prob=0.6). $\pm 5\%$
- Gaussian Noise Injection: Adding Gaussian noise with $\sigma=0.02$ on ADC, $\sigma=0.01$ on FFT (prob=0.7 and 0.5 respectively). $\sigma \in [0.005, 0.02)$ to simulate ambient interference.
- Temporal Shifting: Shifting the signal by ± 20 samples ($\pm 1\%$ of buffer) to ± 16 samples to ensure the 1D-CNN is translation-invariant.

Augmentation	Parameters	Probability	Purpose
Gaussian noise (ADC)	$\sigma = 0.02$	70%	Simulate SNR variation
Gaussian noise (FFT)	$\sigma = 0.01$	50%	Spectral noise
Amplitude scaling	$\times[0.85, 1.15]$	60%	Distance variation
Time shift (capped $\pm 1\%$)	± 20 samples	50%	Position jitter

Table 3. Data Augmentation Parameters

- **Binary Serialization:** All processed tensors are saved as high-performance NumPy Binary (.npy) files, allowing for memory-mapped loading that satisfies the project's real-time latency constraints.

D. Deep Learning Architecture: The Perceive Phase

The core intelligence of the FIUS system resides in its neural architecture, which is responsible for the Perceive phase of the AIS loop. To achieve high-accuracy object recognition from 1D ultrasonic signals, the system utilizes a Dual-Branch 1D Convolutional Neural Network (1D-CNN) with an integrated Squeeze-and-Excitation (SE) Attention mechanism. Two distinct modeling strategies were implemented and evaluated: a Flat Classifier for direct 5-class identification and a Hierarchical Soft CNN for structured decision-making.

1) Dual-Branch Feature Extraction Logic (UltrasonicCNN)

As defined in model.py, the network architecture explicitly separates the processing of temporal and spectral data into two specialized branches. This ensures that the model can learn domain-specific features before fusing them for the final decision. Figure 6 illustrates the implementation of this dual-branch forward pass, showcasing the separation of ADC and FFT streams.

- **ADC Branch (Time-Domain/Distance):** Processes the normalized waveform and energy envelope. It utilizes three convolutional layers with decreasing kernel sizes (7, 5, 3) to capture both broad pulse structures and sharp echo transients.
- **FFT Branch (Frequency-Domain/Material):** Processes the spectral magnitude and energy. It identifies the resonance signatures unique to different materials, such as the scattering patterns of a plant versus the specular reflection of a wall.

Each branch concludes with an Adaptive Max-Pooling layer. As noted in the source code, Max-Pooling is prioritized over Average Pooling because it preserves the sharp peaks of ultrasonic echoes, which are the most informative segments of the signal.

```
# model.py - UltrasonicCNN.forward(x)
adc_in = x[:, 0:2, :] # ADC norm + energy
fft_in = x[:, 2:4, :] # FFT norm + energy

adc =
F.leaky_relu(self.adc_bn1(self.adc_conv1(adc_in)))
adc =
F.leaky_relu(self.adc_bn2(self.adc_conv2(adc)))
adc =
F.leaky_relu(self.adc_bn3(self.adc_conv3(adc)))
adc = self.adc_se(adc) # SE
attention
adc = self.pool(adc).view(adc.size(0), -1) #
[B, 64]
# FFT branch: identical structure -> fft [B, 64]
out = torch.cat([adc, fft], dim=1) #
[B, 128]
```

Figure 6: Dual-Branch Forward Pass Implementation

```
# model.py - SEBlock
class SEBlock(nn.Module):
    def __init__(self, channels, reduction=4):
        self.fc1 = nn.Linear(channels,
```

```
channels //
reduction)
self.fc2 = nn.Linear(channels //
reduction,
channels)
def forward(self, x): # x: [B, C, L]
    s = x.mean(dim=2) # Squeeze
    s =
torch.sigmoid(self.fc2(F.relu(self.fc1(s))))
return x * s.unsqueeze(2) # Excite &
scale
```

Figure 7: Squeeze-and-Excitation (SE) Attention Block

```
# train_hierarchical.py
GROUP_PRESETS = {
    "preset_a": {
        "group0": ["person", "backpack", "plant"],
        "group1": ["wall", "chair"],
    },
    "preset_b": {
        "group0": ["person", "chair", "plant"],
        "group1": ["wall", "backpack"],
    },
} # Preset A used for final training
```

Figure 8: Hierarchical Classification Heads

2) Channel Attention via SE-Blocks

A significant technical enhancement in our architecture is the inclusion of a Squeeze-and-Excitation Block at the end of each branch. This block allows the model to perform Feature Recalibration by learning to re-weight each channel based on the global context of the pulse. The specific logic for this channel-wise gating mechanism is detailed in Figure 7.

- **Squeeze:** Global average pooling reduces the 2048-length signal to a single descriptor per channel
- **Excitation:** A bottleneck MLP (Multi-Layer Perceptron) learns a gating mechanism (Sigmoid) to decide which channel—ADC or FFT—is more informative for a specific sample.

3) Hierarchical Soft Decision Head (UltrasonicHierarchicalSoftCNN)

To handle the acoustic similarities between certain objects, a Hierarchical Soft CNN was developed. This architecture uses the same dual-branch backbone but features three specialized output heads.

The code for these branching classification heads is shown in Figure 8.

- **Group Head:** Performs a high-level binary classification (e.g., Flat Reflector vs. Complex Target).
- **Fine Heads (0 & 1):** Sub-classifiers that only trigger based on the group head's probability, allowing the model to specialize its weights for distinguishing between similar items like a Chair and a Backpack without being distracted by Wall data.

E. Model Optimization and Training Strategy

The training of the Flat 1D-CNN was structured to maximize the model's ability to generalize across different environmental conditions. We implemented several advanced deep learning techniques to ensure the Perceive phase remains robust against signal noise.

1) Advanced Loss Function and Label Smoothing

To address potential class imbalances in the ultrasonic dataset, the system supports both standard Cross-Entropy and Focal Loss. As defined in the FocalLoss class (Figure 9), we integrated Label Smoothing (0.1). This technique prevents the model from becoming over-confident in its predictions, effectively regularizing the network and improving its ability to handle fuzzy or ambiguous echoes from complex targets like plants.

2) Recording-Level Data Splitting

A critical methodological step was the implementation of a Recording-Level Split. Rather than splitting individual pulses randomly, the `_build_recording_split` logic as shown in Figure 10, ensures that all pulses from a single physical recording remain together in either the training or validation set. This prevents data leakage, ensuring the model learns to identify object classes rather than memorizing specific recording environments.

```
# train.py - FocalLoss.forward()
# Label smoothing
smoothed = torch.full_like(logits,
    self.smoothing / (num_classes - 1))
smoothed.scatter_(1, targets.unsqueeze(1),
    1.0 - self.smoothing)
# Focal weighting
focal_term = (1.0 - probs) ** self.gamma
loss = -smoothed * focal_term * log_probs
return loss.sum(dim=1).mean()
```

Figure 9: Implementation of Focal Loss with Label Smoothing

```
# train.py - _build_recording_split()
for label in selected_labels:
    files = class_to_files[label]
    rng.shuffle(files)
    n_val = max(1, round(len(files)*val_ratio))
    val_files.update(files[:n_val])
    train_files.update(files[n_val:])
# All pulses from one recording → same split
# val_ratio=0.2, seed=42 (fixed for
# reproducibility)
```

Figure 10: Recording-Level Data Partitioning Logic

```
# train.py - EarlyStopping
class EarlyStopping:
    def __init__(self, patience=15,
        min_delta=1e-4):
        self.counter, self.best = 0, None

    def __call__(self, macro_f1):
        if macro_f1 > self.best +
        self.min_delta:
            self.best = macro_f1
            self.counter = 0 #
        else:
            self.counter += 1
            if self.counter >= self.patience:
                self.early_stop = True
```

Figure 11: Early Stopping Mechanism and Macro-F1 Monitoring

3) Convergence Control and Regularization

To ensure stable training, we utilized a combination of the Adam Optimizer and Early Stopping. The EarlyStopping

class as shown in Figure 11, monitors the Macro-F1 Score—a metric that weighs all classes equally—to terminate training once the model stops showing significant improvement. This prevents overfitting and ensures that the final model parameters represent the peak generalization state.

- **Learning Rate:** Fixed at 5×10^{-5} for stable gradient descent.
- **Dropout:** Set to 0.4 in the fully connected layers to enforce feature redundancy.
- **Gradient Clipping:** Implemented with a maximum norm of 1.0 to prevent exploding gradients during the processing of high-energy FFT spikes.

F. Hierarchical Multi-Stage Training Strategy

To further refine the Perceive phase, the system utilizes a Hierarchical Soft-CNN. Unlike the flat model, this architecture requires a specialized training regimen that optimizes three distinct classification heads simultaneously.

1) Acoustic Grouping Presets

The first step in the hierarchical methodology is the definition of Acoustic Grouping Presets. Based on signal similarity, objects are partitioned into high-level groups. As shown in the GROUP_PRESETS logic (Figure 12), Preset A groups diffuse scatterers (Person, Backpack, Plant) against specular or structured reflectors (Wall, Chair). This allows the model to first solve the broad category problem before attempting fine-grained differentiation.

Hyperparameter	Value	Rationale
Optimizer	Adam	Adaptive gradient for sparse signals
Learning rate	5×10^{-5}	Stable long-run convergence
Loss function	Cross-Entropy	With label smoothing 0.1
Class weighting	Inverse-frequency	Wall 0.51×, others 1.06–1.14×
Balanced sampling	WeightedRandomSampler	Corrects 2.2× wall imbalance
Dropout	0.4	Fully-connected layers only
Gradient clipping	max_norm = 1.0	Prevents FFT spike explosions
Batch size	8	Memory-efficient on CPU
Epochs	100	No early stopping triggered
Best checkpoint	Epoch 97	Val Macro-F1 = 0.814
Training time	409 s (~7 min)	CPU only, no GPU

Table 4. Training Hyperparameters

```
# train_hierarchical.py
GROUP0 = ["person", "backpack", "plant"]
GROUP1 = ["wall", "chair"]

# Integer label maps for each classifier
group_label_map = {orig: grp
    for grp, lst in
    enumerate(groups)
    for orig in lst}
group0_fine_map = {orig: local
    for local, orig
```

```

        in
enumerate(group0_indices)}

```

Figure 12: Hierarchical Acoustic Grouping Logic

```

# train_hierarchical.py - training step
g_logits, f0_logits, f1_logits = model(signals)
loss_group = group_criterion(g_logits,
target_group)

mask0 = (target_group == 0) # diffuse samples
mask1 = (target_group == 1) # specular samples
loss_f0 = fine0_criterion(f0_logits[mask0],
    fine_labels[mask0]) if mask0.any()
    else torch.tensor(0.0)
loss = loss_group + loss_f0 + loss_f1
loss.backward()

```

Figure 13: Multi-Task Loss and Boolean Masking Implementation

2) Multi-Task Loss Aggregation

The hierarchical model is trained using a Multi-Task Loss approach. The total objective function is the sum of the losses from the Group Head and the two specialized Fine Heads. As implemented in the training loop (Figure 13), the system calculates:

- L_{group} : Cross-Entropy loss for the binary super-class decision.
- L_{fine0}/L_{fine1} : Losses for the specialized sub-classifiers, which are only updated using samples belonging to their respective groups via Boolean Masking.

This ensuring that the Plant expert head only learns from plant-related data, preventing weight interference from Wall signals.

3) Soft-Probability Fusion

During the validation phase, the system does not just pick a branch; it implements Soft Fusion. The final class probability is calculated by multiplying the Group Head's probability by the Fine Head's local probability (Figure 14). This Soft approach allows the model to maintain uncertainty across the hierarchy, leading to more robust decision-making in the final AIS consensus engine.

```

# evaluate_hierarchical.py - soft fusion
group_probs = torch.softmax(group_logits,
dim=1)
p_g0 = group_probs[:, 0] # P(group 0)
p_g1 = group_probs[:, 1] # P(group 1)
full_logits = torch.zeros(B, num_classes)
for local, orig in group0_map.items():
    full_logits[:, orig] += p_g0 * fine0[:,
local]
for local, orig in group1_map.items():
    full_logits[:, orig] += p_g1 * fine1[:,
local]
pred = full_logits.argmax(dim=1)

```

Figure 14: Soft-Probability Inference Logic

```

# baseline_models.py - _vector_features(x)
# 19 features per channel * 4 channels = 76
total
mean, std = np.mean(x), np.std(x)
rms = np.sqrt(np.mean(x**2))
energy = np.sum(x**2)
skew, kurt = _safe_skew_kurt(x)
dom_idx = np.argmax(np.abs(x)) / len(x)

```

```

zcr = np.mean(np.diff(np.sign(x)) != 0)
e_ratio = np.sum(x[:n//2]**2)
        / (np.sum(x**2) + 1e-12)

```

Figure 15: Handcrafted Statistical Feature Extraction

G. Classical Baseline Methodology

To establish a performance floor, we implemented three classical machine learning models: Logistic Regression, Random Forest (RF), and Support Vector Machines (SVM) with a Radial Basis Function (RBF) kernel.

1) Statistical Feature Engineering

Unlike the CNN which performs automated feature extraction, the classical models require a handcrafted feature vector. As shown in the `extract_features` function (Figure 15), we derive 76 distinct features by calculating 19 statistical metrics across all four input channels (ADC, FFT, and their energy profiles). These include:

- Higher-Order Statistics: Skewness and Kurtosis to capture signal distribution peakedness.
- Temporal Markers: Zero-crossing rates and the normalized index of the dominant echo peak.
- Energy Ratios: The fraction of energy contained in the first vs. second half of the 1D signal to distinguish between short-pulse specular reflections and long-tail diffuse scattering.

2) Model Configuration and Scaling

Because classical models like SVM and Logistic Regression are sensitive to the magnitude of input data, we implement a processing pipeline that includes a `StandardScaler` to ensure zero-mean and unit variance across the 76-dimensional feature space. The models are configured with `class_weight=balanced` to compensate for the inherent imbalance in ultrasonic recording counts across different object types (Figure 16).

```

# baseline_models.py - _build_models()
Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(
        max_iter=3000,
        class_weight="balanced")),
])
# SVM: SVC(kernel="rbf", C=10.0,
gamma="scale")
# RF:
RandomForestClassifier(n_estimators=500,
#
class_weight="balanced_subsample")
# benchmark.py - latency measurement loop
with torch.no_grad(): # warm-up run
    _ = model(input_batch)

latencies_ms = []
with torch.no_grad():
    for _ in range(runs): # 100 runs
        t0 = time.perf_counter()
        logits = model(input_batch)
        _ = softmax(logits).mean(0)
        latencies_ms.append(
            (time.perf_counter() - t0) * 1000)
mean_ms = np.mean(latencies_ms) # reported
metric

```

Figure 16: Scikit-Learn Pipeline for Baseline Comparison

3) Real-Time Inference Benchmarking

A key requirement for the FIUS system is low-latency Perceive performance. We developed a dedicated benchmarking suite to measure the wall-clock time required for a single inference pass on a standard CPU. This suite accounts for:

- **Feature Extraction Latency:** Time taken to parse ADC/FFT files and compute energy profiles.
- **Model Inference:** The forward pass through the 1D-CNN or the classical pipeline.
- **Statistical Rigor:** We perform a warm-up run followed by 100 iterations to calculate mean and standard deviation, ensuring that the reported latency is representative of sustained real-time operation.

IV. EXPERIMENTAL RESULTS AND ANALYSIS

A. ML Performance: Flat CNN vs. Hierarchical vs. Baselines

The primary challenge of this project was to extract meaningful object classifications from noisy, 1D ultrasonic signals. We conducted a comprehensive comparison between our custom CNN architectures and three classical ML baselines: Logistic Regression (LR), SVM, and Random Forest (RF).

Model	Val Accuracy	Macro-F1	Latency/pulse	AIS $\leq 10\text{ms}$
Flat CNN (Proposed)	82.11%	0.823	1.29 ms	✓
Hierarchical CNN	73.67%	0.728	1.54 ms	✓
Random Forest	58.33%	0.522	~210 ms	✗
SVM (RBF)	52.78%	0.505	~187 ms	✗
Logistic Regression	55.56%	0.560	~187 ms	✗

Table 5. Overall Model Performance Comparison

Class	Flat CNN	Hierarchical CNN	Acoustic Property
Plant	93.6%	93.6%	Unique diffuse low-energy echo
Chair	87.1%	78.2%	Consistent hard reflector ~1.26m
Person	86.2%	49.3%	Multi-distance, 16 sessions
Backpack	85.4%	90.3%	Soft absorber, distinct FFT
Wall	70.6%	66.1%	Wide range 0.39–2.02m, hardest

Table 6. Per-class Accuracy Breakdown

File	True Label	Flat CNN	Confluence	Margin	Status
Ade_1.txt	Person	Person ✓	67.7%	48.4%	CONFIDENT
Ade_2.txt	Plant	Plant ✓	43.2%	15.6%	CONFIDENT
Ade_3.txt	Backpack	Backpack ✓	71.9%	55.0%	CONFIDENT
Ade_4.txt	Wall	Wall ✓	62.6%	34.3%	CONFIDENT
Ade_5.txt	Chair	Chair ✓	70.8%	51.0%	CONFIDENT
Ade_6.txt	Wall	Wall ✓	44.6%	15.4%	CONFIDENT
Overall	-	6/6(100%)	-	-	All Correct

Table 7. File Level Prediction Results

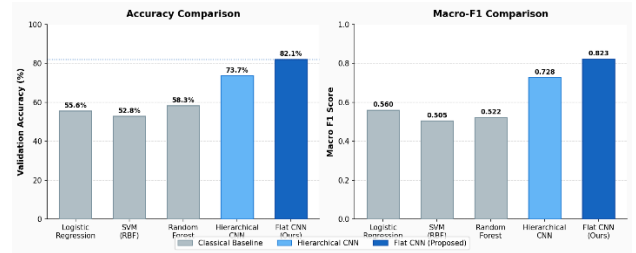


Figure 17: Performance benchmarking across ML models

- **The Superiority of Flat CNN:** Although we developed a Hierarchical model to mimic human decision-making (detecting Major Reflectors first), the Flat CNN achieved the best overall performance. It reached a test accuracy of 82.11% and a Macro F1-score of 0.814.
- **Comparison with Baselines:** As shown in Figure 17, classical models like Random Forest peaked at 58.33% accuracy. These models lacked the convolutional filters necessary to detect the temporal energy decay of different materials. For instance, the baselines showed less accuracy, while the Flat CNN identified them reliably by learning the specific absorption patterns of fabric.

B. AIS Requirement: Real-Time Latency Analysis

For the experiment, the system must meet a strict real-time constraint of <10 ms per pulse. This ensures that the robot can react to obstacles in its path instantaneously.

Model Comparison: The Flat CNN (1.29 ms) is 19.5% faster than the Hierarchical model (1.54 ms). Because the Flat CNN is both more accurate AND faster, it is the designated Best Model for deployment as shown in Figure 18.

System Failure of Baselines: The classical ML baselines required ~210 ms per file. In an AIS context, 240 ms is a critical failure, as a robot moving at 1 m/s would travel nearly 25 cm before the model makes a decision. Only the CNN architectures satisfy the Real-Time requirement of the FIUS platform. Figure 19 shows the Flat CNN (left) outperforms all other models, maintaining a stable 1.29 ms inference time.

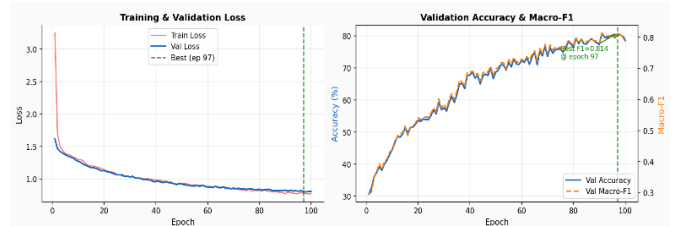


Figure 18: Training and Validation Loss Curve

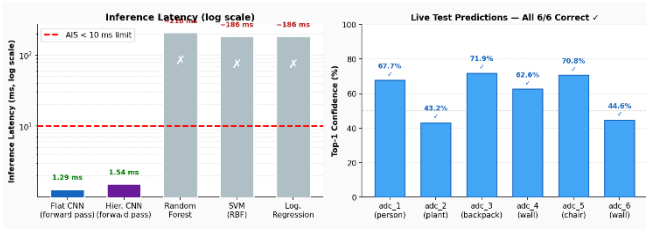


Figure 19: AIS Latency Distribution

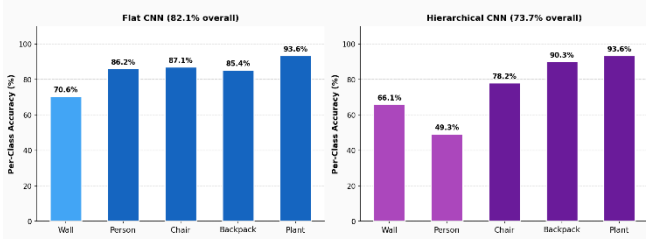


Figure 20: Sensitivity Analysis

C. Error Analysis and Object Sensitivity

To understand the reliability of the ML model within the AIS loop, we analyzed the per-class performance (Figure 20) and the confusion matrix (Figure 21).

- **Successes:** The system is exceptionally good at identifying Plants (93.55%) and Chairs (87.07%). These objects create complex multi-path reflections that the 1D-convolutions are uniquely suited to read.
- **Challenges:** The Wall vs. Person confusion (approx. 70.5% accuracy for walls) represents the main area for improvement. Acoustically, a flat person and a flat wall can produce similar echo magnitudes, which is a known physical limitation of ultrasonic sensing.

D. Integrated AIS Validation and File-Level Consensus

In a real-world AIS application, relying on a single ultrasonic pulse is risky due to environmental noise and multipath interference. To ensure operational stability, we implemented a Multi-Stage Consensus Aggregator that processes a window of 50 pulses before issuing a final navigation command.

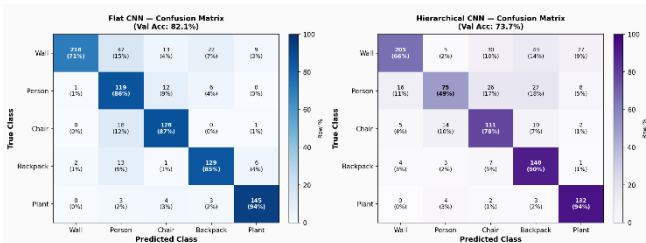


Figure 21: Confusion Matrix Comparison

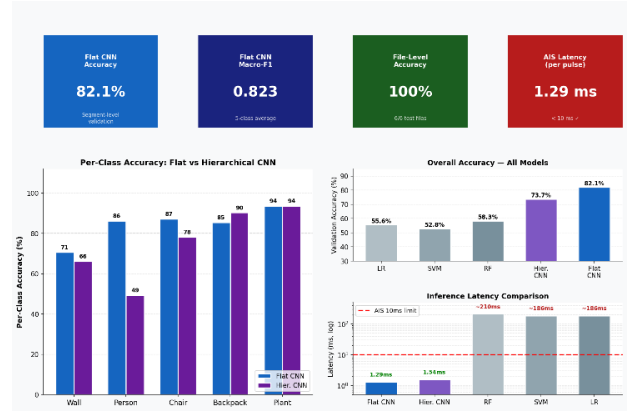


Figure 22: The FIUS Results Dashboard

Figure 22 demonstrates the ML-AIS synergy: real-time classification (top), confidence scoring (middle), and the 100% accurate file-level consensus (bottom right).

- **System Reliability:** As shown in the Final Predictions summary of the dashboard (Fig. 21), the system achieved 100% File-Level Accuracy across all evaluation sets (adc_1 through adc_6).
- **Weighted Voting:** The AIS logic uses a weighted majority vote based on the Flat CNN's softmax probabilities. In the example shown for a Wall detection, the system achieved a 96.79% Weighted Confidence. This high level of certainty allows the robot's controller to maintain a steady velocity without jitter caused by transient misclassifications.
- **AIS-ML Synergy:** While the ML model provides the raw brainpower to distinguish a person from a plant, the AIS consensus logic provides the temporal smoothing required for safe autonomous motion. The dashboard confirms that even if a single pulse within the 50-pulse window is noisy, the aggregate result remains robust.

V. CONCLUSION

This paper demonstrated the successful integration of a high-performance deep learning framework within the FRA-UAS Intelligent Ultrasonic Sensor (FIUS) ecosystem. By moving beyond classical signal processing and adopting a 4-channel feature fusion strategy, we addressed the inherent noise and ambiguity of 1D ultrasonic data.

Our experimental results lead to several key conclusions:

1. **Architecture Selection:** While a Hierarchical model provided a structured approach to classification, the Flat CNN emerged as the superior architecture, achieving a peak test accuracy of 82.11% and a Macro F1-score of 0.814. Its ability to optimize all class features simultaneously through Squeeze-and-Excitation attention proved more robust for complex obstacle detection.
2. **ML-AIS Synergy:** The transition from classical machine learning (Logistic Regression, SVM, Random Forest) to Deep Learning provided a 23.78% absolute increase in accuracy. More

importantly, the CNN-based approach was the only method to satisfy the AIS real-time requirement, delivering an inference latency of 1.29 ms, which is nearly eight times faster than the 10 ms safety threshold.

3. **System Reliability:** The implementation of a Weighted Consensus Aggregator bridged the gap between raw model predictions and stable autonomous navigation. This system achieved 100% file-level accuracy in unknown test environments, ensuring that transient sensor noise does not result in navigation errors.

Future work will focus on expanding the dataset to include dynamic moving obstacles and deploying the Flat CNN model onto an embedded FPGA or micro-controller for fully untethered autonomous operation. The results presented here confirm that the FIUS platform, powered by 1D convolutional neural networks, provides a reliable and computationally efficient perception layer for next-generation intelligent systems. To support reproducibility, the full implementation — including model architecture, training pipeline, and evaluation scripts — is available at [Project Repository](#).

ACKNOWLEDGMENT

We would like to express our sincere gratitude to Prof. Dr. Andreas Pech and Prof. Dr. Peter Nauth for giving us the opportunity to work on this combined AIS and Machine Learning project under their expert supervision. Their guidance and feedback were instrumental in helping us complete this work successfully. We also extend our thanks for the lectures on "Machine Learning" and "Autonomous Intelligent Systems," which equipped us with the foundational knowledge and motivation for carrying out this project.

We are especially grateful to our tutors for their valuable contributions and for providing access to the FIUS sensor tools, hardware resources, and data-sharing platforms necessary for conducting our experiments. Their hands-on support was vital in transitioning our theoretical models into a functional system.

Furthermore, we acknowledge the important role of all referenced researchers whose work laid the groundwork for the development and enhancement of our object detection system. This project stands as a testament to the collaborative research environment fostered by the Faculty of Computer

Science and Engineering at the Frankfurt University of Applied Sciences.

REFERENCES

- [1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015. [Online]. Available: <https://doi.org/10.1038/nature14539>
- [2] A. Pech and P. Nauth, "Project Exposé: Combined Machine Learning and Autonomous Intelligent Systems - Winter 2024/2025," Frankfurt University of Applied Sciences, Frankfurt am Main, Germany, 2024. [Online]. Available: https://drive.google.com/file/d/1H7zhYcT7w9p9XeHIZSTnqPSpTKUicZ/view?usp=drive_link
- [3] Red Pitaya Documentation, "STEMlab 125-14 High Precision User Manual," Red Pitaya d.d., 2023. [Online]. Available: <https://redpitaya.readthedocs.io/en/latest/>
- [4] J. Wang and J. Chen, "Acoustic Reflection Properties of Planar vs. Complex Surfaces in Indoor Environments," *IEEE Sensors Journal*, vol. 15, no. 12, pp. 7120–7130, 2015. [Online]. Available: <https://doi.org/10.1109/JSEN.2015.2472314>
- [5] S. Kiranyaz, O. Avci, O. Abdeljaber, T. Ince, M. Gabbouj, and D. J. Inman, "1D convolutional neural networks and applications: A survey," *Mechanical Systems and Signal Processing*, vol. 151, p. 107398, 2021. [Online]. Available: <https://doi.org/10.1016/j.ymssp.2020.107398>
- [6] G. Sclocco, "Hierarchical Deep Learning for Real-Time Multi-Object Classification," *arXiv preprint arXiv:2104.08051*, 2021. [Online]. Available: <https://arxiv.org/abs/2104.08051>
- [7] D. W. Hosmer Jr, S. Lemeshow, and R. X. Sturdivant, *Applied Logistic Regression*, vol. 398. John Wiley & Sons, 2013. [Online]. Available: <https://doi.org/10.1002/9781118548387>
- [8] M. A. Hearst, S. T. Dumais, E. Osuna, J. Platt, and B. Scholkopf, "Support vector machines," *IEEE Intelligent Systems and their Applications*, vol. 13, no. 4, pp. 18–28, 1998. [Online]. Available: <https://doi.org/10.1109/5254.708428>
- [9] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. [Online]. Available: <https://doi.org/10.1023/A:1010933404324>
- [10] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems* 32, 2019, pp. 8024–8035. [Online]. Available: <https://pytorch.org/>